

Universidad del Istmo de Guatemala

[Curso] · [Catedrático]

Alta disponibilidad en el acceso a una base de datos

Clúster PostgreSQL de tres nodos con Patroni, etcd y HAProxy
sobre macOS (ARM) y Linux (x86) con replicación síncrona y failover automático

INFORME TÉCNICO

Didvin Nohel Estrada Pineda
Carné 14092

Guatemala, octubre de 2026

Alta disponibilidad en el acceso a una base de datos

Autor	Didvin Nohel Estrada Pineda
Carné	14092
Repositorio	github.com/NohelEstradaP/ha_db_ip
Documento complementario	`MANUAL.md` (manual técnico de instalación y operación)

Resumen

Se diseñó, implementó y probó un sistema de **alta disponibilidad** para una base de datos PostgreSQL 16 distribuida en **tres equipos físicos** con sistemas operativos y arquitecturas de procesador distintos:

Nodo	Equipo	Sistema	CPU
n1	MacBook Air M3	macOS	ARM64
n2	PC	Pop!_OS (Linux)	x86-64
n3	MacBook Pro	macOS	ARM64

Los equipos se comunican por una red privada en malla (Tailscale).

Componentes de la solución:

- **Patroni** administra cada instancia de PostgreSQL y automatiza el failover.
- **etcd** aporta el consenso distribuido (algoritmo Raft) y el quórum de 2 de 3 que evita el *split-brain*.
- La **replicación síncrona** garantiza que toda transacción confirmada exista en al menos dos equipos (RPO = 0).
- **HAProxy** y una cadena de conexión libpq con varios hosts hacen que el cliente siempre encuentre al primario.

Una carga constante de transferencias bancarias (180 cambios por minuto por generador, el doble del mínimo exigido) se mantuvo mientras se provocaron fallas reales:

Escenario	Tiempo sin escrituras
Apagado brusco del primario	26,1 s
Detención limpia del primario	3,0 s
Partición de red del primario	22 s hasta el nuevo primario; el nodo aislado se degradó solo a los 11 s
Caída de una réplica	0 s
Desconexión de 10 minutos	0 s; re-sincronización en 5 s
Reinstalación completa de un nodo	0 s; reincorporación automática en 24 s

En todas las pruebas la suma de saldos se mantuvo exactamente en Q1,000,000.00 y no se perdió ni se duplicó ninguna transacción confirmada. La verificación final encontró los mismos 1,301 movimientos en los tres nodos que el cliente había recibido como confirmados, con hashes idénticos.

Índice

1. Introducción
2. Objetivos
3. Marco teórico
4. Análisis del problema y requerimientos
5. Diseño de la solución
6. Arquitectura
7. Implementación
8. Metodología de pruebas
9. Resultados
10. Incidentes encontrados durante la implementación
11. Análisis de resultados
12. Limitaciones y trabajo futuro
13. Conclusiones
14. Referencias
15. Anexos

1. Introducción

En la mayoría de sistemas de información la base de datos es el componente más crítico: si deja de responder, todas las aplicaciones que dependen de ella se detienen, y si falla en el momento equivocado puede perder transacciones ya reportadas como exitosas. Una base de datos instalada en un único equipo es, por definición, un **punto único de falla**. Basta un corte de energía, un reinicio por actualizaciones, un disco dañado o un cable desconectado.

La **alta disponibilidad** (*High Availability*, HA) busca que el servicio siga funcionando aunque fallen componentes individuales. En bases de datos relacionales esto implica resolver tres problemas a la vez:

1. **Redundancia de datos:** mantener copias actualizadas en más de un equipo.
2. **Detección y conmutación automática:** decidir sin intervención humana qué copia toma el control cuando falla la principal, y redirigir a los clientes hacia ella.
3. **Consistencia:** garantizar que en ningún momento existan dos copias aceptando escrituras independientes (*split-brain*), que nada confirmado se pierda y que un equipo que regresa se reincorpore sin corromper los datos.

Este informe documenta una solución completa a estos tres problemas, construida con herramientas de código abierto usadas en producción, desplegada en tres equipos físicos heterogéneos y validada con fallas provocadas de forma deliberada.

2. Objetivos

2.1 Objetivo general

Implementar el acceso de alta disponibilidad a una base de datos que reciba al menos 90 cambios por minuto, replicada en tres equipos, de forma que la pérdida de cualquiera de ellos no interrumpa el servicio ni comprometa la integridad de los datos, y que el equipo afectado se re-sincronice automáticamente al regresar.

2.2 Objetivos específicos

1. Desplegar un clúster PostgreSQL de tres nodos en tres equipos físicos con macOS y Linux, y con arquitecturas ARM64 y x86-64.
2. Automatizar la detección de fallas y la promoción de un nuevo primario mediante Patroni y un almacén de consenso (etcd).
3. Garantizar cero pérdida de transacciones confirmadas (RPO = 0) mediante replicación síncrona.
4. Impedir el *split-brain* mediante quórum.
5. Medir el tiempo de recuperación (RTO) en distintos escenarios de falla.
6. Comprobar la integridad de los datos con invariantes verificables antes, durante y después de cada falla.
7. Automatizar la instalación, la reincorporación de nodos y el monitoreo para que el sistema sea reproducible.

3. Marco teórico

3.1 Write-Ahead Log (WAL)

PostgreSQL no escribe los cambios directamente en los archivos de tablas. Primero los registra de forma secuencial en el **Write-Ahead Log** (WAL), un registro de bitácora en el que cada modificación queda descrita a nivel de bytes en páginas de datos. Una transacción se considera confirmada (durable) cuando su registro de `COMMIT` llegó al WAL en disco. Ante una caída, PostgreSQL reproduce el WAL desde el último *checkpoint* y reconstruye el estado exacto. Cada posición del WAL se identifica con un **LSN** (*Log Sequence Number*), por ejemplo `0/46467A8`.

3.2 Replicación física por streaming

Si el WAL describe todos los cambios, basta enviarlo a otro servidor para que este los reproduzca y mantenga una copia idéntica. Eso es la **replicación física por streaming**:

1. La réplica se crea con una copia base del primario (`pg_basebackup`).
2. La réplica abre una conexión de replicación (*walreceiver*) y el primario le envía el WAL (*walsender*) en tiempo real.
3. La réplica aplica el WAL continuamente y acepta consultas de solo lectura (*hot standby*).

La copia es **byte a byte**, no lógica. Por eso primario y réplicas deben usar la misma versión mayor de PostgreSQL y un formato de datos compatible.

3.3 Replicación síncrona y asíncrona

- **Asíncrona:** el primario confirma el `COMMIT` al cliente apenas lo escribe en su propio disco. Si muere antes de que el WAL llegue a la réplica, esa transacción confirmada **se pierde** al promover la réplica.
- **Síncrona:** el primario espera a que al menos una réplica confirme que escribió el WAL en su disco antes de responder al cliente (`synchronous_commit = on` y `synchronous_standby_names`). Toda transacción confirmada existe en **dos** equipos. El costo es la latencia de un viaje de red por `COMMIT` y la necesidad de contar siempre con una réplica síncrona disponible.

3.4 RPO y RTO

- **RPO** (*Recovery Point Objective*): cuántos datos se pueden perder en una falla. Con replicación síncrona, RPO = 0 para transacciones confirmadas.
- **RTO** (*Recovery Time Objective*): cuánto tiempo pasa sin poder escribir hasta que el servicio se restablece.

3.5 Consenso distribuido, Raft y etcd

Para decidir cuál nodo es el primario no basta con que cada nodo "crea" algo: deben **ponerse de acuerdo**. **etcd** es un almacén clave-valor distribuido que implementa el algoritmo de consenso **Raft**:

- Los miembros eligen un líder de etcd. Toda escritura se replica y solo se confirma cuando la **mayoría** de los miembros la registró.
- Con N miembros, la mayoría es $\lfloor N/2 \rfloor + 1$ y se toleran $\lfloor (N - 1)/2 \rfloor$ fallas. Con 3 miembros, la mayoría es 2 y se tolera **una** falla.
- etcd ofrece **leases** (claves con tiempo de vida): una clave asociada a un lease desaparece si nadie lo renueva antes de que expire.

3.6 Patroni

Patroni es un agente escrito en Python que corre junto a cada PostgreSQL y lo administra: lo inicia, lo configura como primario o réplica, lo promueve y lo degrada. Funciona con un ciclo (*HA loop*) que se ejecuta cada `loop_wait` segundos:

1. El primario renueva en etcd la **llave de líder** (`/service/hadb/leader`), que tiene un TTL (`ttl`).
2. Si el primario muere o queda aislado, deja de renovarla y la llave expira al cumplirse el TTL.
3. Las réplicas detectan que no hay líder, comparan su posición de WAL y la más adecuada intenta tomar la llave. etcd garantiza que **solo una** lo logre.
4. El ganador se promueve (`pg_promote`) y los demás se reconfiguran para replicar desde él.
5. Si un primario pierde el contacto con etcd y no puede renovar su llave, **se degrada a sí mismo** a réplica de solo lectura.

En **modo síncrono** (`synchronous_mode: true`), Patroni guarda además en etcd cuál réplica es la síncrona y **solo esa** puede ser promovida, porque es la única con garantía de tener todo lo confirmado.

Patroni expone una API REST (puerto 8008). `GET /primary` responde 200 solo en el primario y `GET /replica` responde 200 solo en las réplicas sanas. Esa API la usan los balanceadores.

3.7 Timelines y pg_rewind

Cada vez que una réplica es promovida, PostgreSQL abre un **timeline** nuevo (1 → 2 → 3…), una nueva rama de la historia del WAL. El primario anterior puede haber escrito en el timeline viejo un poco de WAL que nunca llegó a la réplica: son transacciones no confirmadas al cliente si la replicación es síncrona. Para reincorporarlo como réplica hay que descartar esa divergencia. **`pg_rewind`** lo hace sin copiar toda la base: compara ambos servidores desde el último punto común y copia solo los bloques modificados. Requiere `wal_log_hints = on` o *data checksums*.

3.8 Slots de replicación y retención de WAL

Un **slot de replicación** hace que el primario retenga el WAL que una réplica todavía no recibió. Junto con `wal_keep_size`, esto permite que una réplica desconectada un tiempo se ponga al día solo con el WAL, sin volver a copiar toda la base.

3.9 Split-brain y teorema CAP

El **split-brain** ocurre cuando dos nodos se consideran primarios a la vez y aceptan escrituras independientes. Los datos divergen y no se pueden reconciliar automáticamente. Con dos nodos no hay forma de distinguir "el otro murió" de "se cortó la red". Con tres y quórum sí: el lado con mayoría continúa y el lado en minoría sabe que está aislado.

El **teorema CAP** establece que, ante una partición de red, un sistema distribuido debe elegir entre **consistencia** y **disponibilidad**. Esta solución elige **consistencia**: el lado en minoría deja de aceptar escrituras y el lado con mayoría sigue disponible.

3.10 Enrutamiento de clientes

- **libpq con varios hosts:** la biblioteca cliente de PostgreSQL acepta `host=a,b,c` y `target_session_attrs=read-write`. Prueba los hosts en orden y se queda con el que acepta escrituras.
- **HAProxy:** balanceador TCP que, con chequeos HTTP a la API de Patroni, envía el puerto 5000 solo al primario y el 5001 a las réplicas.

4. Análisis del problema y requerimientos

4.1 Requerimientos del enunciado

#	Requerimiento	Interpretación técnica
R1	Al menos 90 cambios por minuto	Carga sostenida y medible de `INSERT` / `UPDATE`, también durante las fallas
R2	Acceso desde dos o más equipos y siempre algún mirror	Clientes en varios equipos; al menos una copia actualizada en todo momento
R3	Uno de los equipos se apagará	Cualquiera, incluido el primario, sin aviso
R4	Integridad y funcionalidad continúan con un equipo desconectado	Failover automático, sin pérdida ni duplicación de datos confirmados
R5	Al restablecer la comunicación se vuelve a sincronizar	Reincorporación automática como réplica, incluso del primario anterior

4.2 Restricciones del entorno

- Tres equipos personales: dos MacBook con Apple Silicon y una PC con Pop!_OS. Durante el proyecto el tercer equipo resultó ser una segunda Mac en lugar de la segunda PC Linux prevista.
- Los equipos pueden estar en la misma red o en redes distintas (casa, universidad).
- No hay servidores dedicados ni IP públicas, y la demostración se hace en vivo.

5. Diseño de la solución

5.1 Alternativas evaluadas

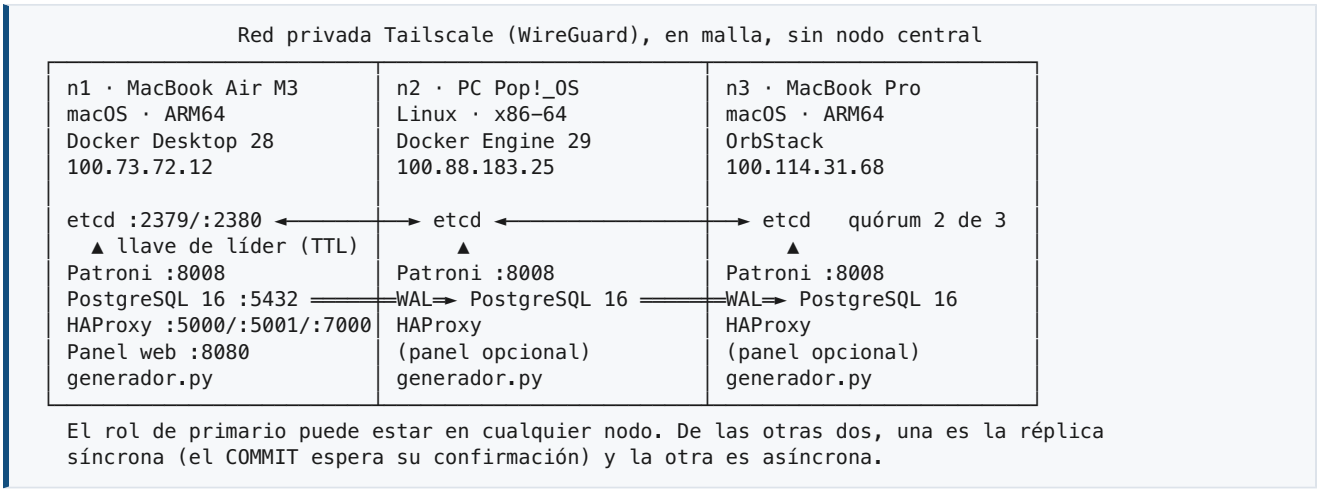
Alternativa	Ventajas	Desventajas	Decisión
PostgreSQL + Patroni + etcd	Estándar de la industria; failover automático; replicación síncrona; `pg_rewind`; API de salud	Más componentes	**Elegida**
MariaDB Galera	Multi-maestro síncrono	En macOS solo por Docker; *certification conflicts* bajo contención	Descartada
MongoDB Replica Set	Failover nativo y simple	NoSQL; el curso trata bases relacionales	Descartada
Replicación nativa sin orquestador	Simple	Failover manual: no cumple R4	Descartada

5.2 Decisiones de diseño

1. **Tres nodos.** Es el mínimo que permite quórum (2 de 3). Con dos nodos el split-brain es inevitable ante una partición.
2. **Replicación síncrona con una réplica síncrona** (`synchronous_mode: true`, `synchronous_node_count: 1`) y la otra asíncrona. Así se tiene RPO = 0 sin que la caída de una réplica detenga las escrituras: si cae la síncrona, Patroni nombra síncrona a la otra.
3. **`synchronous_mode_strict: false`.** Si solo quedara el primario con quórum, preferir disponibilidad. No ocurre con una sola falla, porque etcd perdería el quórum antes.
4. **Docker en los tres equipos con la misma imagen** (`postgres:16-bookworm` + Patroni 4.0.4). La replicación física copia archivos byte a byte, así que los tres nodos deben usar el mismo binario de PostgreSQL y la misma biblioteca de colación. Con una instalación nativa, macOS usaría sus propias colaciones y Linux las de glibc, con riesgo de corromper índices de texto.
5. **`initdb --locale=C.UTF-8` y `--data-checksums`.** El orden de texto queda independiente de la plataforma y los *checksums* detectan corrupción y habilitan `pg_rewind`.
6. **Tailscale** como red. Cada equipo recibe una IP fija `100.x.y.z` que no cambia aunque cambie la red física, y la red es en malla, sin un equipo central que sea punto único de falla.
7. **Tiempos de Patroni ajustados:** `ttl` de 20 s, `loop_wait` de 5 s y `retry_timeout` de 5 s, cumpliendo `loop_wait + 2·retry_timeout ≤ ttl`. El failover queda en unos 20–30 s ante un apagado brusco; con los valores por defecto serían unos 40 s.
8. **Tiempos de etcd para WAN:** `heartbeat-interval` de 250 ms y `election-timeout` de 2,500 ms. Toleran la latencia de Tailscale entre redes distintas sin elecciones falsas.
9. **Cliente idempotente.** Cada transacción lleva un identificador único `(generador, seq)` y se reintenta con `ON CONFLICT DO NOTHING`, lo que da semántica de "exactamente una vez" aunque la conexión se caiga en pleno `COMMIT`.

6. Arquitectura

6.1 Vista general



6.2 Componentes y versiones

Componente	Versión	Función	Puertos
PostgreSQL	16.15 (Debian 12, pgdg)	Motor de base de datos	5432
Patroni	4.0.4 (psycopg 3, etcd v3)	Orquestación, failover, `pg_rewind`, API REST	8008
etcd	3.5.17	Consenso Raft, llave de líder, estado síncrono	2379 cliente · 2380 pares
HAProxy	3.0	5000 → primario · 5001 → réplicas · 7000 → estadísticas	5000, 5001, 7000
Tailscale	1.10x	VPN en malla con IP fija por equipo	UDP 41641
Docker	Desktop 28.0.4 · Engine 29.9.2 · OrbStack	Contenedores con la misma imagen en los 3 nodos	—
Panel web	propio (Python + HTML)	Estado en tiempo real y cambio de primario	8080

6.3 Flujo de una escritura

1. El cliente abre una conexión con `host=n1,n2,n3 target_session_attrs=read-write`. libpq descarta las réplicas y se queda con el primario.
2. El cliente ejecuta la transacción: bloquea las dos cuentas, inserta el movimiento y actualiza los saldos.
3. En el `COMMIT`, el primario escribe el WAL en su disco y lo envía a ambas réplicas.
4. El primario **espera** la confirmación de la réplica síncrona y solo entonces responde "COMMIT" al cliente.
5. La réplica asíncrona aplica el WAL milisegundos después.

6.4 Flujo de un failover (apagado brusco del primario)

Tiempo	Evento
t = 0 s	El primario deja de existir. Los `COMMIT` en curso no reciben respuesta y los clientes detectan la conexión muerta (keepalives / `tcp_user_timeout` ≈ 10 s).
0–20 s	La llave de líder sigue vigente en etcd hasta que se cumple su TTL (20 s desde la última renovación). Las escrituras quedan en pausa y los clientes reintentan.

Tiempo	Evento
≈ 20–25 s	La llave expira. Patroni en las réplicas detecta la ausencia de líder. Solo la **réplica síncrona** es elegible y toma la llave en etcd, con quórum de los 2 nodos vivos.
≈ 26 s	La réplica ejecuta `pg_promote`, abre un timeline nuevo y empieza a aceptar escrituras. La otra réplica pasa a replicar de ella y Patroni la nombra síncrona.
≈ 26 s +	libpq y HAProxy encuentran al nuevo primario. Los clientes reintentan la misma `seq` (idempotente) y recuperan el atraso.
Al volver	El primario viejo detecta que su timeline quedó atrás. Patroni ejecuta `pg_rewind` contra el nuevo primario y lo arranca como réplica.

6.5 Comportamiento ante una partición de red

Si el primario pierde la red pero sigue encendido:

- **Lado aislado:** no puede renovar la llave de líder porque no tiene quórum de etcd. Patroni lo degrada a sí mismo (`demoting self because DCS is not accessible and I was a leader`). Además, con replicación síncrona, ningún `COMMIT` puede confirmarse en ese lado, porque la réplica síncrona es inalcanzable.
- **Lado con mayoría:** cuando expira la llave, promueve a la réplica síncrona.
- **Resultado:** nunca hay dos primarios aceptando escrituras confirmadas.

7. Implementación

7.1 Estructura del repositorio

Ruta	Contenido
`instalar.sh`	Prepara un equipo como nodo (macOS y Pop!_OS)
`docker-compose.yml`	Servicios del nodo: etcd, Patroni/PostgreSQL y HAProxy
`.env.example`	Plantilla de identidad del nodo, IP y contraseñas
`patroni/Dockerfile`, `patroni/patroni.yml`	Imagen PostgreSQL 16 + Patroni y su configuración
`haproxy/haproxy.cfg`	Enrutamiento por rol según la API de Patroni
`sql/esquema.sql`	Tablas `cuentas` y `movimientos`
`app/generador.py`	Generador de carga idempotente
`app/verificar.py`	Verificador de integridad entre los 3 nodos
`app/monitor.py`	Monitor de terminal
`app/panel.py`, `app/panel.html`	Panel web en tiempo real con cambio de primario
`scripts/`	Iniciar, estado, crear BD, carga, panel, patronictl, prueba de puertos, secretos
`laboratorio/`	Los 3 nodos en una sola máquina, para pruebas

7.2 Imagen de Patroni

```
FROM postgres:16-bookworm
RUN apt-get update \
 && apt-get install -y --no-install-recommends python3 python3-venv curl \
 && python3 -m venv /opt/patroni \
 && /opt/patroni/bin/pip install --no-cache-dir "patroni[etcd3,psycopg3]==4.0.4" \
 && apt-get clean && rm -rf /var/lib/apt/lists/*
ENV PATH="/opt/patroni/bin:${PATH}"
COPY patroni.yml /etc/patroni/patroni.yml
USER postgres
ENTRYPOINT ["patroni", "/etc/patroni/patroni.yml"]
```

La imagen base es multi-arquitectura: en las Mac se construye para ARM64 y en la PC para x86-64, con la misma versión de PostgreSQL (16.15) y de glibc.

7.3 Configuración de Patroni

```
scope: hadb
namespace: /service/
bootstrap:
  dcs:
    ttl: 20
    loop_wait: 5
    retry_timeout: 5
    maximum_lag_on_failover: 1048576      # no promover réplicas con más de 1 MB de atraso
    synchronous_mode: true
    synchronous_mode_strict: false
    synchronous_node_count: 1
    postgresql:
      use_pg_rewind: true
      use_slots: true
      parameters:
```

```
        wal_level: replica
        hot_standby: "on"
        wal_log_hints: "on"
        max_wal_senders: 10
        max_replication_slots: 10
        wal_keep_size: 1GB
        synchronous_commit: "on"
        password_encryption: scram-sha-256
  initdb: [ { encoding: UTF8 }, { locale: C.UTF-8 }, data-checksums ]
postgresql:
  listen: 0.0.0.0:5432
  data_dir: /var/lib/postgresql/data/patroni
  pg_hba:
    - local   all          all                      trust
    - host    replication  replicator  0.0.0.0/0     scram-sha-256
    - host    all          all         0.0.0.0/0     scram-sha-256
```

Los datos que cambian por nodo (nombre, IP anunciada, endpoints de etcd y contraseñas) llegan por variables de entorno `PATRONI_*` desde `docker-compose.yml`, de modo que el archivo es idéntico en los tres nodos.

7.4 Servicios por nodo (extracto de `docker-compose.yml`)

```
services:
  etcd:
    image: gcr.io/etcd-development/etcd:v3.5.17
    command:
      - --name=${NODE_NAME}
      - --advertise-client-urls=http://${NODE_IP}:2379
      - --initial-advertise-peer-urls=http://${NODE_IP}:2380
      - --initial-cluster=n1=http://${N1_IP}:2380,n2=http://${N2_IP}:2380,n3=http://${N3_IP}:2380
      - --initial-cluster-state=${ETCD_ESTADO_INICIAL:-new}
      - --heartbeat-interval=250
      - --election-timeout=2500
  patroni:
    build: ./patroni
    init: true
    stop_grace_period: 60s
    environment:
      PATRONI_NAME: ${NODE_NAME}
      PATRONI_ETCD3_HOSTS: "${N1_IP}:2379,${N2_IP}:2379,${N3_IP}:2379"
      PATRONI_POSTGRESQL_CONNECT_ADDRESS: ${NODE_IP}:5432
      PATRONI_RESTAPI_CONNECT_ADDRESS: ${NODE_IP}:8008
  haproxy:
    image: haproxy:3.0
```

Todos los servicios usan `restart: unless-stopped`. Junto con el arranque automático de Docker, un equipo que se enciende vuelve solo al clúster.

7.5 Enrutamiento con HAProxy

```
listen primario
    bind *:5000
    option httpchk
    http-check send meth GET uri /primary
    http-check expect status 200
    default-server inter 2s fall 2 rise 2 on-marked-down shutdown-sessions
    server n1 "${N1_IP}:5432" check port 8008
    server n2 "${N2_IP}:5432" check port 8008
    server n3 "${N3_IP}:5432" check port 8008
```

Cada 2 s HAProxy pregunta a la API de Patroni de cada nodo. Solo el primario responde 200 en `/primary`. Con `on-marked-down shutdown-sessions` se cortan de inmediato las conexiones a un nodo que deja de ser primario, para que el cliente reconecte al nuevo.

7.6 Cadena de conexión de la aplicación

```
host=100.73.72.12,100.88.183.25,100.114.31.68 port=5432 dbname=hadb user=app
target_session_attrs=read-write connect_timeout=3
keepalives=1 keepalives_idle=5 keepalives_interval=2 keepalives_count=3 tcp_user_timeout=10000
```

`keepalives` y `tcp_user_timeout` hacen que una conexión a un nodo que murió sin cerrar el socket se detecte en unos 10 s, en lugar de quedar colgada indefinidamente.

7.7 Automatización

- **`instalar.sh`.** Detecta el sistema operativo y prepara el equipo:
- instala Homebrew/apt y Docker, o usa OrbStack si ya está instalado;
- instala Tailscale y desactiva la suspensión;
- verifica los puertos libres y detecta el Receptor AirPlay de macOS;
- genera el `.env`, prepara Python y construye las imágenes.

Se puede volver a ejecutar y, al terminar, ofrece levantar el nodo.

- **`scripts/iniciar.sh`.** Levanta el nodo, espera el quórum de etcd, el primario y la sincronización del propio nodo. Si el nodo fue reinstalado (sin datos de etcd) y el clúster ya existe, retira el miembro viejo de etcd, lo vuelve a agregar y lo arranca como miembro `existing`. El modo de arranque se guarda en `.env` para que las siguientes ejecuciones no recreen los contenedores.
- **`scripts/crear_bd.sh`.** Crea el usuario, la base y el esquema en el primario, de forma idempotente.
- **`scripts/carga.sh` y `scripts/panel.sh --fondo`.** Ejecutan el generador y el panel en segundo plano.

7.8 Panel web en tiempo real

`app/panel.py` es un servidor HTTP que solo usa la biblioteca estándar de Python y `psycopg`:

- **Un hilo por nodo** consulta cada segundo la API de Patroni (`/patroni`), la salud de etcd (`/health`), las estadísticas de HAProxy (CSV del puerto 7000) y el propio PostgreSQL (conteo de movimientos y suma de saldos).
- **Otro hilo** escribe y lee por el mismo camino que la aplicación (la cadena de varios hosts), para medir los cambios por minuto y detectar las pausas de escritura.
- **Un agregador** arma una foto por segundo y detecta eventos: caídas, cambios de rol, cambios de timeline y la duración de cada failover medida desde el lado del cliente.

`app/panel.html` presenta sin dependencias externas:

- indicadores, tarjetas por nodo y un diagrama de replicación;
- una gráfica de cambios por minuto con la línea del mínimo (90) y marcas de failover;
- los eventos, los generadores y los últimos movimientos.

Cada tarjeta incluye el botón **"Hacer primario"**, que pide un *switchover* planificado a la API de Patroni (`POST /switchover`). Como Patroni solo promueve a la réplica síncrona, si el destino no lo es el panel encadena los pasos necesarios (por ejemplo n2 → n1 → n3) y muestra el avance.

8. Metodología de pruebas

8.1 Modelo de datos e invariantes

```
CREATE TABLE cuentas (
    id integer PRIMARY KEY,
    saldo numeric(14,2) NOT NULL CHECK (saldo >= 0), ...);
CREATE TABLE movimientos (
    id bigserial PRIMARY KEY,
    generador text NOT NULL, seq bigint NOT NULL,
    origen integer REFERENCES cuentas(id), destino integer REFERENCES cuentas(id),
    monto numeric(14,2), primario text, creado timestamptz DEFAULT now(),
    UNIQUE (generador, seq));
-- 100 cuentas × Q10,000.00 = Q1,000,000.00
```

Cada operación del generador es una **transferencia** en una sola transacción:

- bloquea ambas cuentas en orden de `id` (`SELECT … FOR UPDATE`), lo que evita *deadlocks*;
- inserta el movimiento con `ON CONFLICT (generador, seq) DO NOTHING`;
- descuenta del origen y abona al destino.

Son **3 cambios por transacción**: un generador a 1 transacción por segundo produce 180 cambios por minuto.

Invariantes verificados:

1. **Conservación:** `SUM(saldo) = 1,000,000.00`. Una transacción aplicada a medias o dos veces alteraría la suma.
2. **Continuidad:** para cada generador, `MAX(seq) = COUNT(*)`. Ninguna transacción confirmada falta.
3. **Igualdad entre nodos:** con la carga detenida, el md5 de las dos tablas es idéntico en los 3 nodos (`verificar.py --esperar` espera a que las réplicas alcancen el LSN del primario).
4. **Cliente contra base:** cada transacción que el generador registró en su CSV como confirmada existe en la base.

8.2 Idempotencia del cliente

Si la conexión se cae durante el `COMMIT`, el cliente no sabe si la transacción quedó guardada. El generador:

- reconecta y **reintenta la misma `seq`**: si ya existía, `ON CONFLICT` la ignora y el resultado se registra como `duplicado`;
- lleva un ritmo fijo y, tras una pausa, ejecuta las transacciones atrasadas para que el promedio por minuto se mantenga.

8.3 Cómo se provocaron las fallas

Falla	Método	Equivale a
Apagado brusco de un nodo	`docker kill etcd patroni haproxy` (SIGKILL a los 3 contenedores)	Corte de energía: nada se cierra ordenadamente
Detención limpia	`docker compose stop patroni`	Apagado o reinicio ordenado del equipo
Partición de red	`tailscale down` en el nodo	Cable o WiFi desconectado con el equipo encendido
Cambio planificado	`patronictl switchover` o el botón del panel	Mantenimiento programado
Reinstalación	`docker compose down -v` (borra todos los datos del nodo)	Equipo formateado

Antes de cada prueba se confirmó el clúster sano (1 Leader, 1 Sync Standby, 1 Replica con lag 0) y la carga activa.

8.4 Validación previa en laboratorio

El mismo diseño se probó primero con los 3 nodos en un solo equipo (`laboratorio/`, una red Docker con IP fijas), usando la misma imagen y los mismos archivos de configuración:

Prueba	Resultado
Failover por apagado brusco (4 repeticiones)	22–26 s
Partición del primario	27 s, sin dos primarios
Reingreso del primario viejo	`pg_rewind` correcto
Verificación de integridad	OK en todas las pruebas

9. Resultados

9.1 Compatibilidad entre arquitecturas (ARM64 ↔ x86-64)

Paso	Resultado
Arranque del clúster con n3 (Mac, ARM64) como primario	n2 (PC, x86-64) copió la base con `pg_basebackup` y replicó en streaming
Switchover n3 → n1 → n2	n2 (x86-64) primario; las dos Mac (ARM64) replicaron desde él
Datos en el primario x86	100 cuentas, suma 1,000,000.00

```
PostgreSQL 16.15 (Debian 16.15-1.pgdg12+2) on x86_64-pc-linux-gnu    (n2)
PostgreSQL 16.15 (Debian 16.15-1.pgdg12+2) on aarch64-unknown-linux-gnu (n3)
```

La replicación física funciona en ambas direcciones entre arquitecturas porque los tres nodos comparten versión, formato de datos (64 bits, *little-endian*, `MAXALIGN` 8) y colación (`C.UTF-8`).

9.2 Pruebas de falla en los equipos reales

Condiciones: generador a 1 transacción por segundo (180 cambios por minuto), 23 de octubre de 2026.

Prueba	Escenario	Nodo (rol)	Escrituras en pausa (RTO)	Nuevo primario	Suma	Faltan	Re-sincronización
P8	Switchover planificado	n2 (primario)	**3,0 s**	n1	✓	0	3 nodos sanos en 14 s
P9	Detención limpia de Patroni	n1 (primario)	**3,0 s**	n3	✓	0	réplica en 3 s
P3	**Apagado brusco**	n1 (primario)	**26,1 s**	n3	✓	0	`pg_rewind` desde n3, sano en 45 s
P2	Apagado brusco	n1 (réplica)	**0 s**	—	✓	0	streaming en 4 s
P4	**Partición de red**	n1 (primario)	se degradó solo a los **11 s**; promoción a los **22 s**	n3	✓	0	réplica en timeline 11, lag 0
P7	10 min desconectado con carga	n1 (réplica)	**0 s**	—	✓	0	600 transacciones recuperadas por WAL en **5 s**
—	Caída real no planificada (reinicio de Docker en n2)	n2 (primario)	**5,0 s**	n1	✓	0	n2 volvió solo como réplica
—	**Reinstalación completa** (`down -v`)	n1 (réplica)	**0 s**	—	✓	0	reincorporación automática a etcd y copia base: **24 s**

Durante P3, con n1 caído dos minutos, el clúster se mantuvo en estado "Degradado: 2 de 3 nodos" escribiendo a **180 cambios por minuto**, con suma 1,000,000.00 y 0 faltantes. Durante P7 se escribieron 600 transacciones (1,800 cambios) sin n1.

9.3 Línea de tiempo de la partición (P4)

Hora (UTC)	Evento	Fuente
01:30:11	`tailscale down` en n1 (primario)	prueba
01:30:22	`n1: ERROR: Error communicating with DCS · demoting self because DCS is not accessible and I was a leader`	log de Patroni en n1
01:30:33	Timeline 10 abierto en **n3** (nuevo primario)	`patronictl history`
reconexión	n1 vuelve como réplica en el timeline 11, lag 0	`patronictl list`

El nodo aislado dejó de ser primario **antes** de que el otro lado promoviera uno nuevo: en ningún instante hubo dos primarios.

9.4 Cambio de primario desde el panel

Cambio solicitado	Pasos ejecutados	Tiempo total
n3 → n2 (n2 era la réplica síncrona)	n3 → n2	10,1 s
n2 → n3 (n3 no era la síncrona)	n2 → n1 → n3	22,3 s
n3 → n2	n3 → n2	16,3 s
n1 → n2 (n2 no era la síncrona)	2 pasos	26,3 s

El panel rechazó correctamente las solicitudes inválidas: el nodo ya era primario, el nodo no existía o había un cambio en curso.

9.5 Verificación final de integridad

Con la carga detenida, después de todas las pruebas:

```
nodo  rol      cuentas           suma     movs faltan  md5 cuentas  md5 movs
n1    réplica      100   1,000,000.00     1301      0  7a6e1a907591 ea098493b542
n2    réplica      100   1,000,000.00     1301      0  7a6e1a907591 ea098493b542
n3    PRIMARIO     100   1,000,000.00     1301      0  7a6e1a907591 ea098493b542

RESULTADO: OK. 3 nodos con datos idénticos, suma = 1,000,000, sin secuencias faltantes, un solo
primario.
```

- **Cliente contra base:** el CSV del generador registró **1,301 transacciones confirmadas** y la base tiene exactamente **1,301 movimientos**, con `seq` 1 a 1,301 sin huecos ni repeticiones.
- **Reintentos:** 8 transacciones necesitaron un segundo intento, una por cada cambio de primario, y todas quedaron guardadas una sola vez:

Hora	seq	Latencia	Escrita en	Causa
19:22:53	74	1,7 s	n1	P8 switchover
19:23:20	102	0,8 s	n3	P9 detención limpia
19:24:14	153	3,9 s	n2	switchover de preparación
19:24:26	167	1,8 s	n1	switchover de preparación
19:25:24	198	29,1 s	n3	**P3 apagado brusco**
19:29:30	467	5,5 s	n2	switchover de preparación

Hora	seq	Latencia	Escrita en	Causa
19:29:45	486	1,8 s	n1	switchover de preparación
19:31:09	515	56,9 s	n3	**P4 partición** (el cliente estaba del lado aislado)

Al cierre de la sesión de pruebas, la base acumuló **2,485 movimientos** de 3 corridas de generador, con suma 1,000,000.00 y 0 secuencias faltantes.

10. Incidentes encontrados durante la implementación

#	Síntoma	Causa	Solución aplicada
1	El build de la imagen falla con `403 Forbidden` en los repositorios de Debian	El proxy de una red bloqueaba `deb.debian.org` por HTTP	Cambiar de red. El caso quedó documentado en la guía de problemas
2	Tres servicios del laboratorio fallan con `image already exists`	Compose construía la misma imagen 3 veces en paralelo	Construir una vez antes de levantar (`lab.sh`)
3	macOS reporta los puertos 5000 y 7000 ocupados	El **Receptor AirPlay** (`ControlCenter`) escucha en ellos	Desactivarlo; el instalador ahora lo detecta y lo explica
4	`pip` corta la descarga de `psycopg` (`Read timed out`)	Red lenta: pip corta a los 15 s por defecto	`--timeout 120 --retries 10` en el instalador
5	Riesgo de que etcd no forme quórum	Las IP de n2 y n3 quedaron invertidas en el `.env` de una Mac	Corrección del `.env`; la guía exige comparar `grep _IP .env` en los 3
6	El tercer equipo tenía OrbStack en lugar de Docker Desktop	El instalador habría instalado Docker Desktop encima	El instalador detecta y usa OrbStack
7	`switchover` responde `412 candidate name does not match with sync_standby`	En modo síncrono Patroni solo promueve a la réplica síncrona	Switchover sin `--candidate`; el panel encadena pasos
8	Aparente diferencia de 1 fila entre el CSV y la base	El generador lanzado en segundo plano ignoraba SIGINT y seguía escribiendo	Detenerlo con SIGTERM (`scripts/carga.sh detener`); la cifra cuadró exacta
9	Un nodo reinstalado no puede unirse a etcd (`cluster ID mismatch`)	Un etcd sin datos intenta crear un clúster nuevo	`iniciar.sh` retira y vuelve a agregar el miembro y arranca como `existing`
10	Volver a correr `iniciar.sh` recreaba etcd y Patroni	El modo de arranque de etcd cambiaba entre ejecuciones	El modo se guarda en `.env`
11	La conexión SSE del panel dejaba de actualizar con varias pestañas	Límite de 6 conexiones simultáneas por servidor en el navegador	Consulta periódica cada segundo

11. Análisis de resultados

11.1 Cumplimiento de requerimientos

#	Requerimiento	Evidencia	Cumple
R1	≥ 90 cambios/min	180 cambios/min medidos con un generador, también con un nodo caído (con un generador por equipo, la capacidad de diseño es de 540)	✓
R2	Acceso desde ≥ 2 equipos y siempre un mirror	Clientes en los 3 equipos; una réplica síncrona en todo momento; cada `COMMIT` en 2 equipos	✓
R3	Un equipo se apaga	Probado con el primario y con réplicas, de forma brusca, limpia, por red y por reinstalación	✓
R4	Integridad y funcionalidad continúan	Escrituras restablecidas en 3–26 s; suma constante; 0 faltantes; 0 duplicados; cliente = base	✓
R5	Re-sincronización al volver	`pg_rewind` (primario viejo), WAL (réplica), copia base automática (reinstalación)	✓

11.2 RTO

- **Caídas limpias o planificadas: unos 3–5 s.** Patroni libera la llave de líder al detenerse, así que la réplica síncrona se promueve en el siguiente ciclo.
- **Apagado brusco: unos 26 s.** El TTL de 20 s cuenta desde la última renovación de la llave, más hasta un `loop_wait` (5 s) para detectarlo y el tiempo de promoción.
- **Bajar el RTO** es posible reduciendo `ttl`, por ejemplo a 10 s con `loop_wait` de 3 s. A cambio aumenta el riesgo de failovers innecesarios por cortes breves de red, que es especialmente relevante entre redes distintas por Internet.

11.3 RPO

RPO = 0 en todas las pruebas. La replicación síncrona hizo que cada transacción confirmada existiera en dos nodos, y el modo síncrono de Patroni solo promovió réplicas que la tenían. Las transacciones "en vuelo" durante la caída nunca fueron confirmadas al cliente: este las reintentó y quedaron guardadas exactamente una vez.

11.4 Disponibilidad ante partición

La prueba P4 muestra el comportamiento CP (consistencia y tolerancia a particiones) esperado. El cliente que estaba en el lado aislado esperó 56,9 s hasta la reconexión: es el precio de no permitir escrituras sin quórum. Los clientes del lado con mayoría continuaron a los 22 s.

11.5 Carga durante las fallas

La ventana móvil de 60 s se mantuvo por encima de 90 cambios por minuto. Tras cada pausa, el generador ejecuta las transacciones atrasadas y produce un pico visible en la gráfica, de hasta unos 1,800 cambios por minuto durante algunos segundos, que compensa el tiempo sin escrituras.

12. Limitaciones y trabajo futuro

Limitación	Impacto	Mejora propuesta
Con 3 nodos se tolera una sola falla	Con 2 caídas el nodo restante queda en solo lectura (correcto, pero sin servicio de escritura)	5 nodos para tolerar 2 fallas
Sin TLS entre componentes	El tráfico va cifrado por WireGuard (Tailscale), pero PostgreSQL, etcd y la API de Patroni no usan TLS propio	Certificados para PostgreSQL, etcd y la API REST
Sin respaldos externos ni PITR	Un error lógico (`DELETE` accidental) se replica a todos los nodos	pgBackRest con WAL archiving a almacenamiento externo
Sin *watchdog* de hardware	Un primario "congelado" (no caído) depende solo del TTL	`/dev/watchdog` (softdog) en Linux
Panel sin autenticación	Cualquiera en la red Tailscale puede cambiar el primario	Autenticación o `--solo-lectura` en paneles compartidos
RTO de unos 26 s ante caída brusca	Pausa perceptible de escrituras	Ajustar `ttl` / `loop_wait` según la estabilidad de la red
Monitoreo propio	Sin alertas ni historial persistente	Prometheus + Grafana + Alertmanager

13. Conclusiones

1. **La solución cumple los cinco requerimientos.** Con PostgreSQL 16, Patroni y etcd en tres equipos, la base de datos siguió aceptando escrituras ante la pérdida de cualquier equipo, por apagado brusco, detención limpia, partición de red o reinstalación completa, y cada equipo se re-sincronizó sin intervención al regresar.
2. **La integridad se demostró, no solo se asumió.** Los invariantes de negocio (suma constante de saldos), la continuidad de secuencias, los hashes idénticos entre nodos y la conciliación exacta entre lo que el cliente recibió como confirmado y lo que hay en la base (1,301 = 1,301) prueban RPO = 0 y la ausencia de duplicados.
3. **El quórum es la pieza que hace seguro el failover automático.** En la partición, el primario aislado se degradó a los 11 s, antes de que el lado con mayoría promoviera otro a los 22 s, así que nunca existieron dos primarios.
4. **La replicación síncrona y la idempotencia del cliente se complementan.** La primera garantiza que lo confirmado no se pierda; la segunda, que lo no confirmado pueda reintentarse sin duplicarse.
5. **Los contenedores permitieron un clúster heterogéneo.** macOS con ARM64 y Linux con x86-64 replicaron en ambas direcciones gracias a una imagen única y a la colación `C.UTF-8`.
6. **La automatización es parte de la disponibilidad.** El instalador, la reincorporación automática de nodos (24 s tras borrar todos los datos de un nodo) y el panel reducen el tiempo y el error humano en la operación diaria y en la recuperación.

14. Referencias

1. PostgreSQL Global Development Group. *PostgreSQL 16 Documentation*. Capítulos 27 (*High Availability, Load Balancing, and Replication*), 30 (*Reliability and the Write-Ahead Log*) y la referencia de `pg_rewind`. https://www.postgresql.org/docs/16/
2. Patroni. *Patroni Documentation*: modo síncrono, parámetros del DCS, API REST. https://patroni.readthedocs.io/
3. etcd Authors. *etcd Documentation v3.5*: clustering, runtime reconfiguration, tuning. https://etcd.io/docs/v3.5/
4. Ongaro, D. y Ousterhout, J. (2014). *In Search of an Understandable Consensus Algorithm (Raft)*. USENIX ATC.
5. Brewer, E. (2000). *Towards Robust Distributed Systems* (teorema CAP). PODC.
6. HAProxy Technologies. *HAProxy 3.0 Configuration Manual*. https://docs.haproxy.org/3.0/configuration.html
7. PostgreSQL Global Development Group. *libpq: Connection Strings* (`target_session_attrs`, conexiones con varios hosts). https://www.postgresql.org/docs/16/libpq-connect.html
8. Tailscale Inc. *Tailscale Documentation*. https://tailscale.com/kb

15. Anexos

Anexo	Archivo
A. Manual técnico de instalación y operación	`MANUAL.md` / `MANUAL.pdf`
B. Plan de implementación detallado	`PLAN_ALTA_DISPONIBILIDAD.md` / `.pdf`
C. Guía de puesta en marcha y pruebas	`GUIA_PENDIENTES.md` / `.pdf`
D. Casos límite analizados	`EDGE_CASES.md`
E. Configuración completa	`docker-compose.yml`, `patroni/`, `haproxy/`, `sql/esquema.sql`
F. Programas	`app/generador.py`, `app/verificar.py`, `app/monitor.py`, `app/panel.py`
G. Evidencias (logs de Patroni, CSV de generadores, verificaciones)	carpeta `evidencias/` del nodo n1